Logica do jogo - Oficina Arena / Arena Bot

Disciplina: Computacao Grafica (integracao com PP01 de IA)
Objetivo: mapa da logica para melhorar o gameplay sem reescrever Minimax/A*.

---

1. Duas camadas

Camada | Onde | Papel
Regra de jogo + IA | `world/core` + `world/ia` (copia do Arena Bot) | Decide alvo, rota, coleta, pontuacao
Cena 3D | `scene/`, `app/`, OpenGL | Edita a grade e desenha o estado

O OpenGL nao decide rota. Ele le posicao/estado e anima.

Fluxo resumido

1. Menu -> Jogar -> selecao de robo (Robo 3 / Robo 5; Alfa = escolhido, Beta = o outro)
2. Modo de jogo -> moeda 3D -> partida
3. Tecla P / R -> `WorldBridge` sincroniza cena -> `Partida`
4. A cada ~0,35 s -> `Partida.executar_turno()`
5. Decidir alvo (Minimax ou base via BC) -> A* -> andar 1 celula
6. Na base: descarregar; senao: tentar coletar se `Acoletar`
7. Espelhar minerios removidos na cena 3D
8. Fim do round com vencedor -> danca da vitoria no centro (1 das 2 dancas do modelo); empate sem danca

---

2. Loop de um turno

Arquivo: `world/core/partida.py` -> `Partida.executar_turno`

1. Alterna Alfa / Beta (`turno % 2`).
2. Se bateria <= 0 -> so passa a vez.
3. Alvo: reusa caminho valido; senao Minimax; se BC tem `Pdescarga` -> forca base.
4. Rota: A* ate o alvo (ate ~n_minerios+3 tentativas).
5. Avanca 1 celula; custo -2 (normal) ou -4 (armadilha).
6. Na base -> `descarregar()` (pontos + bateria cheia).
7. Fora da base -> `tentar_coletar()` so se a BC derivar `Acoletar`.
8. Se chegou no alvo e nao coletou -> limpa alvo e reavalia.
9. Fim: ambos sem bateria, ou sem minerios/entregas possiveis. Vence quem tem mais pontos.

---

3. Mundo

Peca | Regra
Grade | 15x9 por padrao (ajustavel em Configuracoes ate 28x17, o limite do galpao), movimento ortogonal (N/S/L/O)
Parede | Intransitavel
Armadilha | Transitavel, custo dobrado
Minerio | diamante 100 / rubi 80 / ouro 50 / prata 30 / bronze 10; quantidade configuravel por tipo (padrao = minimo base 15x9: 3/4/5/6/7); o minimo sobe com a area da arena (`ceil(base x area/135)`), como a bateria; diamantes: 1 par perto das bases + resto no meio; rubis: 1 par perto + resto aleatorio
Base Alfa | (0, 0)

Base Beta | canto oposto `(largura-1, altura-1)`
Defaults | 25 paredes, 6 armadilhas, 25 minerios na 15x9 (3+4+5+6+7); caps UI: 150 paredes / 80 armadilhas

- Sem colisao robo-robo.
- Decoracao `LIVRE` (cadeira, planta, luminaria) ocupa a cena mas **nao** entra na arena -> na simulacao o robo "atravessa" a malha.

---

4. Estado do robo

Arquivo: `world/core/agente.py`

Campo | Papel
`posicao`, `base` | Celula atual e base propria
`bateria` | Comeca no valor configurado (minimo recomendado sobe com o tamanho da arena; em 15x9 o minimo e 120); -2 / -4 por passo
`carga` | 0...3 espacos; diamante ocupa 2, demais 1
`itens` | contagem de minerios carregados (bonus de entrega = `itensx10`)
`pontuacao` | +valor ao coletar; +`itensx10` ao descarregar
`alvo`, `caminho` | Meta e fila A*
`bc` | Base de conhecimento proposicional

---

5. Tres camadas de decisao

5.1 Base de conhecimento (coleta / retorno)

Arquivo: `world/ia/conhecimento.py`

Observados: `Balta` (bateria > 10), `Ccheia` (carga >= 3), `Sminerio`, `Bcritica`, `Aadj` (observado sem regra).

Regras:

- `Balta /\ ¬Ccheia => Pcoleta`
- `Ccheia => Pdescarga`
- `Bcritica => Pdescarga`
- `Sminerio /\ Pcoleta => Acoletar`

5.2 Minimax a-b (qual minerio / base)

Arquivo: `world/ia/estrategia.py` - profundidade 4.

- Acoes = escolher minerio ou base (nao cada passo).
- Pre-filtro: so minerios com bateria para ida+volta+margem; senao volta a base.
- Avaliacao favorece valor do minerio e vantagem de distancia vs oponente.
- Alfa e Beta usam a **mesma** politica.

5.3 A* (caminho celula a celula)

Arquivo: `world/ia/navegacao.py`

- Heuristica: Manhattan.
- Custo de aresta: 1 (normal) / 2 (armadilha).

---

6. O que a oficina 3D muda

Acao | Efeito
Catalogo | bancada/estante -> `PAREDE`; minerio; armadilha; base; decor -> `LIVRE`
P / Espaco | sync cena -> partida (pause/resume)
R | reinicia a partir da cena
Titulo da janela | placar, bateria, carga, turno; no fim: vencedor e volta a edicao

Gap: minerios colocados pelo usuario viram ouro generico (valor 50) no sync.

Decisao de escopo do projeto: nao reescrever Minimax/A* do PP01; evoluir via ponte cena<->partida, HUD, placement e constantes.

---

7. Alavancas para melhorar

1. Feedback - placar/bateria/carga no titulo (ja feito); overlay OpenGL se quiser.
2. Consistencia cena<->IA - decor como parede ou nao-ocupante; preservar tipo/valor do minerio no sync.
3. Ritmo visual - `turn_interval` alinhado ao passo do robo.
4. Balanceamento - `world/config.py`: custos, carga, pesos Minimax, densidade do mapa.
5. Politicas assimetricas - so se quiser diferenciar Alfa/Beta (cuidado com o escopo de CG).

---

8. Arquivos-chave

Objetivo | Arquivo
Ponte cena<->partida | `world/bridge.py`
Turnos / fim | `world/core/partida.py`
Robo | `world/core/agente.py`
Constantes | `world/config.py`
A* / Minimax / BC | `world/ia/*.py`
Input / titulo | `app/application.py`
Catalogo -> celula | `scene/catalog.py`
Placement | `scene/workshop.py`

Documento gerado a partir do mapa de logica da Oficina Arena.